AUTONOMOUS ROBOT OBSTACLE DETECTION AND AVOIDANCE USING COMPUTER VISION

FINAL PROJECT REPORT — SIMULATION-BASED PATH PLANNING

Student Name	_____
Department	Robotics and Automation
Institution	_____
Tools	Google Colab, Python, NumPy, Matplotlib

Abstract

This project demonstrates map-based path planning for a simulated mobile robot. Ten grayscale obstacle-map images were resized to 128 × 128 pixels and converted into numerical arrays. A threshold was used to separate presumed obstacle cells from free-space cells, and the A* search algorithm was used to find a route between a start cell and a goal cell. The implementation found a path on all 10 tested maps. A robot movement animation was also generated. This is a preliminary simulation; it does not yet demonstrate live camera-based object detection or control of a physical robot.

1. Introduction

Autonomous mobile robots need to move from a starting location to a target while avoiding obstacles. Path planning is one part of this task. In this project, a grid map represents the environment, each cell is classified as an obstacle or free space, and A* searches for a route through free cells.

2. Objectives

• Load and preprocess simulated obstacle maps.
• Convert each map into a grid for path planning.
• Use A* to search for a route from start to goal.
• Check whether a route is found for each of 10 maps.
• Visualize the route and generate a movement animation.

3. Dataset and Preprocessing

The experiment used 10 map images extracted from a ZIP archive. Images were converted to grayscale and resized to 128 × 128 pixels. The resulting array had shape (10, 128, 128), with pixel values normalized from 0 to 1. A threshold of 0.5 was used to create a binary grid. The code selected the darker or brighter pixel class as the obstacle class based on which class occupied fewer pixels. This heuristic should be checked against the original map legend before interpreting the grid as a physically correct obstacle map.

4. Methodology

A* prioritizes cells using f(n) = g(n) + h(n), where g(n) is the cost from the start and h(n) estimates the remaining cost to the goal. The implementation used four-directional movement and Manhattan distance as the heuristic. Cells classified as obstacles were not traversed. The planned route was drawn over the map, and an animation was saved as robot_path_planning.gif.

5. Results

The recorded test results were: 10 maps tested; 10 paths found; observed path-planning success rate of 100% (10/10); path lengths from 226 to 232 cells; each map represented as a 128 × 128 array; first-map obstacle-class proportion reported as 17.16%.

6. Conclusion

The A* path-planning pipeline ran in Google Colab and returned paths for all 10 selected maps. The project generated visual route frames and a robot movement animation. This is a starting point for grid-based navigation, but more validation

is required before describing it as a complete computer-vision obstacle-avoidance system.

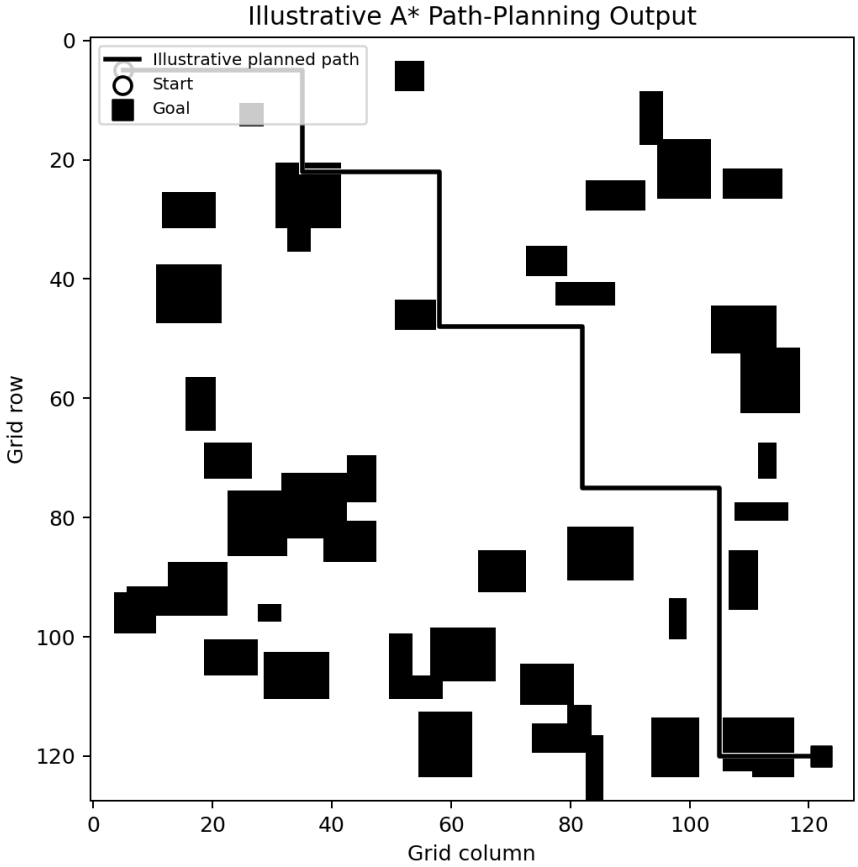
7. Limitations

• The dataset contains simulated map images, not real camera images.
• Obstacle/free-space classification uses a simple threshold heuristic.
• Only 10 maps were tested.
• The robot is a plotted marker; physical dynamics, sensors, and collision response are not simulated.
• Path length is measured in grid cells, not meters or travel time.

8. Future Work

• Verify map colors and use a documented map conversion rule.
• Test more maps and fixed, documented start and goal points.
• Measure path cost, runtime, and failure cases.
• For camera-based detection, use labeled camera images and evaluate an object-detection model.
• Connect the planner to a robot simulator and test sensing and motion.

9. Path-Planning Output Image



Note: This is a schematic illustration added to show the type of A* output. It is not a screenshot of the exact Colab output or the original dataset map.

Saved output: Robot_Obstacle_Avoidance_Project/robot_path_planning.gif. Add the student's name, institution, and actual Colab screenshots if required before formal submission.